

PYTHON

Module 7: Variable Mastery

CODING

Understanding Containers

In Python, variables are essentially names that refer to objects in the computer's memory. They act as containers for storing data values.

Dynamic Creation: Unlike other languages, Python has no command for declaring a variable. A variable is created the moment you first assign a value to it (e.g., `x = 5`).

Flexibility: Python variables are 'Dynamic'. This means they do not need to be declared with any particular type, and can even change type after they have been set.

Example: `x = 4` (integer) can later become `x = "Sally"` (string) without any errors.

Casting and Type Identification

If you need to ensure a variable is of a specific data type, you use 'Casting'.

Common Casting Functions:

- `str()` - Converts to a string.

- `int()` - Converts to an integer.

- `float()` - Converts to a decimal number.

Identifying Types: Use the `type()` function to inspect the data type of a variable at any time. This is a common debugging practice in CodeHive development.

Variable Naming Standards

A variable can have a short name (x) or a descriptive name (total_price). To write professional code, follow these rules:

Legal Rules:

•

- Must start with a letter or underscore (_).

•

- Cannot start with a number.

•

- Can only contain alphanumeric characters and underscores (A-z, 0-9, and _).

•

Case Sensitivity: In Python,

age ,

Age , and AGE are three different variables. Always use consistent naming to avoid bugs.

•

Assigning Multiple Values

Python provides an elegant way to assign values to multiple variables in a single line, making your code more concise.

Techniques:

• Multi-Assignment:

```
x, y, z = "Orange", "Banana", "Cherry" .
```

• One Value to Multiple: `x = y = z = "Apple" .`

Collection Unpacking: If you have a list of values, Python can 'unpack' them into variables: `fruits = ["apple", "banana"]; x, y = fruits .`

Outputting Variables

The `print()` function is the primary way to output variables. You can output multiple items by separating them with a comma.

Concatenation vs. Comma:

- Comma (Recommended):

`print("Age is", x)` –Supports different data types (string + number).

- Plus (+) Operator: `print("Name: " + x)` –Only works for strings. Using `+` with a string and a number will cause a `TypeError`.

Mathematical Output: For numbers, the `+` character acts as an addition operator rather than joining text.

Scope: Global vs. Local

Scope determines where a variable can be accessed within your program.

Global Variables: Created outside of functions. They are available to every part of your code.

Local Variables: Created inside a function. They only exist while that function is running.

Naming Conflicts: If you create a variable with the same name inside a function, it becomes a 'Local' copy, leaving the original 'Global' variable untouched.

Naming Conventions & Best Practices

At CodeHive, we encourage the 'Snake Case' naming convention for variables: `user_login_count` instead of `userLoginCount` .

Naming Tips:

- Use descriptive names: `price_per_unit` is better than `p` .

- Avoid using Python keywords (like `if` , `while` , `print`) as variable names.

- Keep names concise but meaningful.

Practical Exercise & Summary

Testing your understanding is vital for mastery.

Exercise Challenge:

1. Create three variables `a` , `b` , and `c` for your favorite colors.

2. Unpack a list of numbers into `x` and `y` .

3. Try to add a string and a number using a comma—observe how Python handles it.

Mastering variables is the foundation for all logic-based programming.